had to know whether it was supposed to be first-level. (To make things clumsier, Rails couldn't call Module#include directly, because it was a private method—so it had to use a *Dynamic Dispatch* (49) instead. Recent rubies made include public, but we're talking ancient history here.)

At this point in our story, you'd be forgiven for thinking that include-and-extend created more problems than it solved in the first place. This trick forced multiple modules to contain the same boilerplate code, and it failed if you had more than one level of module inclusions. To address these issues, the authors of Rails crafted ActiveSupport::Concern.

ActiveSupport::Concern

ActiveSupport::Concern encapsulates the include-and-extend trick and fixes the problem of chained inclusions. A module can get this functionality by extending Concern and defining its own ClassMethods module:

part2/using_concern.rb

```
require 'active_support'

module MyConcern
  extend ActiveSupport::Concern

  def an_instance_method; "an instance method"; end

  module ClassMethods
    def a_class_method; "a class method"; end
  end
end

class MyClass
  include MyConcern
end

MyClass.new.an_instance_method  # => "an instance method"
MyClass.a_class_method          # => "a class method"
```

In the rest of this chapter I'll use the word "concern" with a lowercase C to mean "a module that extends ActiveSupport::Concern," like MyConcern does in the example above. In modern Rails, most modules are concerns, including ActiveRecord::Validations and ActiveModel::Validations.

Let's see how Concern works its magic.

A Look at Concern's Source Code

The source code of Concern is quite short but also fairly complicated. It defines just two important methods: extended and append_features. Here is extended: